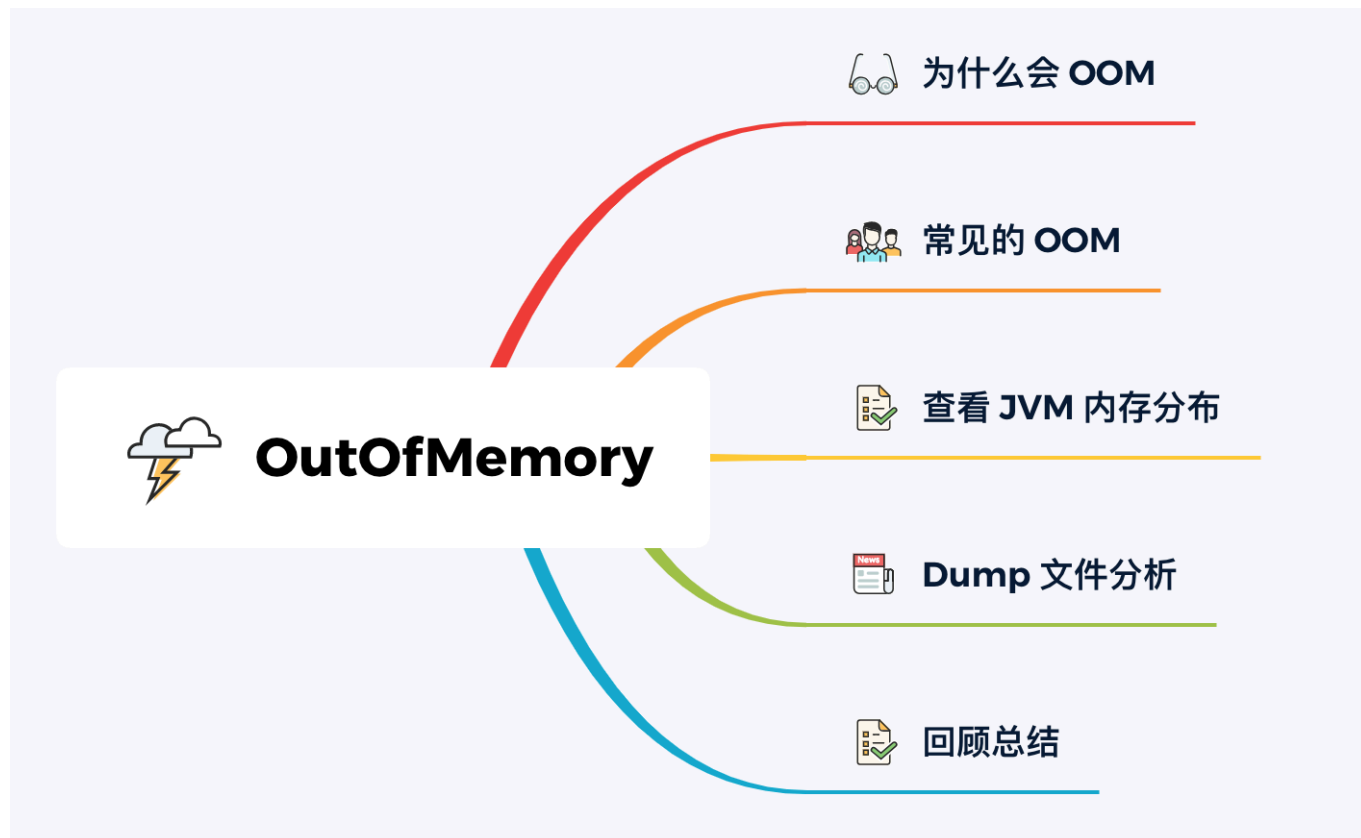


应用出现OOM异常，程序员如何第一时间知道？

OOM 意味着程序存在着漏洞，可能是代码或者 JVM 参数配置引起的。这篇文章和读者聊聊，Java 进程触发了 OOM 后如何排查。

常说对生产环境保持敬畏之心，快速解决问题也是一种敬畏的表现。



为什么会 OOM

OOM 全称 “Out Of Memory”，表示内存耗尽。当 JVM 因为没有足够的内存来为对象分配空间，并且垃圾回收器也已经没有空间可回收时，就会抛出这个错误。

为什么会出现 OOM，一般由这些问题引起：

1. 分配过少：JVM 初始化内存小，业务使用了大量内存；或者不同 JVM 区域分配内存不合理。
2. 代码漏洞：某一个对象被频繁申请，不用了之后却没有被释放，导致内存耗尽。

内存泄漏：申请使用完的内存没有释放，导致虚拟机不能再次使用该内存，此时这段内存就泄露了。因为申请者不用了，而又不能被虚拟机分配给别人用。

内存溢出：申请的内存超出了 JVM 能提供的内存大小，此时称之为溢出。

内存泄漏持续存在，最后一定会溢出，两者是因果关系。

常见的 OOM

比较常见的 OOM 类型有以下几种：

java.lang.OutOfMemoryError: PermGen space

Java7 永久代（方法区）溢出，它用于存储已被虚拟机加载的类信息、常量、静态变量、即时编译器编译后的代码等数据。每当一个类初次加载的时候，元数据都会存放到永久代。

一般出现于大量 Class 对象或者 JSP 页面，或者采用 CgLib 动态代理技术导致。

我们可以通过 `-XX: PermSize` 和 `-XX: MaxPermSize` 修改方法区大小。

Java8 将永久代变更为元空间，报错：`java.lang.OutOfMemoryError: Metadata space`，元空间内存不足默认进行动态扩展

java.lang.StackOverflowError

虚拟机栈溢出，一般是由于程序中存在 **死循环或者深度递归调用** 造成的。如果栈大小设置过小也会出现溢出，可以通过 `-Xss` 设置栈的大小。

虚拟机抛出栈溢出错误，可以在日志中定位到错误的类、方法。

java.lang.OutOfMemoryError: Java heap space

Java 堆内存溢出，溢出的原因一般由于 JVM 堆内存设置不合理或者内存泄漏导致。

如果是内存泄漏，可以通过工具查看泄漏对象到 GC Roots 的引用链。掌握了泄漏对象的类型信息以及 GC Roots 引用链信息，就可以精准地定位出泄漏代码的位置。

如果不存在内存泄漏，就是内存中的对象确实都还必须存活着，那就应该检查虚拟机的堆参数（-Xmx 与 -Xms），查看是否可以将虚拟机的内存调大些。

小结：方法区和虚拟机栈的溢出场景不在本篇过多讨论，下面主要讲解常见的 Java 堆空间的 OOM 排查思路。

查看 JVM 内存分布

假设我们 Java 应用 PID 为 15162，输入命令查看 JVM 内存分布 `jmap -heap 15162`。

```
[xxx@xxx ~]# jmap -heap 15162
Attaching to process ID 15162, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.161-b12

using thread-local object allocation.
Mark Sweep Compact GC

Heap Configuration:
  MinHeapFreeRatio      = 40 # 最小堆使用比例
  MaxHeapFreeRatio      = 70 # 最大堆可用比例
  MaxHeapSize           = 482344960 (460.0MB) # 最大堆空间大小
  NewSize                = 10485760 (10.0MB) # 新生代分配大小
  MaxNewSize            = 160759808 (153.3125MB) # 最大新生代可分配大小
  OldSize               = 20971520 (20.0MB) # 老年代大小
  NewRatio              = 2 # 新生代比例
  SurvivorRatio         = 8 # 新生代与 Survivor 比例
  MetaspaceSize         = 21807104 (20.796875MB) # 元空间大小
  CompressedClassSpaceSize = 1073741824 (1024.0MB) # Compressed Class Space 空间大小限制
  MaxMetaspaceSize      = 17592186044415 MB # 最大元空间大小
  G1HeapRegionSize      = 0 (0.0MB) # G1 单个 Region 大小

Heap Usage: # 堆使用情况
New Generation (Eden + 1 Survivor Space): # 新生代
  capacity = 9502720 (9.0625MB) # 新生代总容量
  used     = 4995320 (4.763908386230469MB) # 新生代已使用
  free     = 4507400 (4.298591613769531MB) # 新生代剩余容量
  52.56726495150862% used # 新生代使用占比
Eden Space:
```

```
capacity = 8454144 (8.0625MB) # Eden 区总容量
used      = 4029752 (3.8430709838867188MB) # Eden 区已使用
free      = 4424392 (4.219429016113281MB) # Eden 区剩余容量
47.665996699370154% used # Eden 区使用占比
From Space: # 其中一个 Survivor 区的内存分布
capacity = 1048576 (1.0MB)
used      = 965568 (0.92083740234375MB)
free      = 83008 (0.07916259765625MB)
92.083740234375% used
To Space: # 另一个 Survivor 区的内存分布
capacity = 1048576 (1.0MB)
used      = 0 (0.0MB)
free      = 1048576 (1.0MB)
0.0% used
tenured generation: # 老年代
capacity = 20971520 (20.0MB)
used      = 10611384 (10.119804382324219MB)
free      = 10360136 (9.880195617675781MB)
50.599021911621094% used

10730 interned Strings occupying 906232 bytes.
```

通过查看 JVM 内存分配以及运行时使用情况，可以判断内存分配是否合理。

另外，可以在 JVM 运行时查看最耗费资源的对象，`jmap -histo:live 15162 | more`。

JVM 内存对象列表按照对象所占内存大小排序。

- instances: 实例数;
- bytes: 单位 byte;
- class name: 类名。

```
[root@single ~]# jmap -histo:live 15162 | more
```

num	#instances	#bytes	class name
1:	452160	10851840	cn.acmenlt.demo.oom.CustomObjTest
2:	52223	5212632	[C
3:	9921	2846128	[Ljava.lang.Object;
4:	9678	1758480	[I
5:	2699	1355936	[B
6:	51936	1246464	java.lang.String
7:	10200	1127768	java.lang.Class
8:	23113	739616	java.util.concurrent.ConcurrentHashMap\$Node
9:	7911	696168	java.lang.reflect.Method
10:	3786	335736	[Ljava.util.HashMap\$Node;
11:	10112	323584	java.util.HashMap\$Node
12:	17804	284864	java.lang.Object
13:	6383	255320	java.util.LinkedHashMap\$Entry
14:	143	217648	[Ljava.util.concurrent.ConcurrentHashMap\$Node;
15:	9561	212960	[Ljava.lang.Class;
16:	2893	162008	java.util.LinkedHashMap
17:	6656	159744	org.jboss.netty.util.HashedWheelTimer\$HashedWheelBucket
18:	2370	151680	org.objectweb.asm.Label
19:	2046	147312	java.lang.reflect.Field
20:	1637	130960	java.lang.reflect.Constructor

明显看到 `CustomObjTest` 对象实例以及占用内存过多。

可惜的是，方案存在局限性，因为它只能排查对象占用内存过高问题。

其中 "[代表数组，例如 "[C" 代表 Char 数组，"[B" 代表 Byte 数组。如果数组内存占用过多，我们不知道哪些对象持有它，所以需要 Dump 内存进行离线分析。

```
jmap -histo:live 执行此命令，JVM 会先触发 GC，再统计信息
```

Dump 文件分析

Dump 文件是 Java 进程的内存镜像，其中主要包括 **系统信息、虚拟机属性、完整的线程 Dump、所有类和对象的状态** 等信息。

当程序发生内存溢出或 GC 异常情况时，怀疑 JVM 发生了 **内存泄漏**，这时我们就可以导出 Dump 文件分析。

JVM 启动参数配置添加以下参数：

- -XX:+HeapDumpOnOutOfMemoryError
- -XX:HeapDumpPath=./ (参数为 Dump 文件生成路径)

当 JVM 发生 OOM 异常自动导出 Dump 文件，文件名称默认格式：

```
java_pid{pid}.hprof
```

上面配置是在应用抛出 OOM 后自动导出 Dump，或者可以在 JVM 运行时导出 Dump 文件。

```
jmap -dump:file=[文件路径] [pid]
```

示例

```
jmap -dump:file=./jvmdump.hprof 15162
```

在本地写一个测试代码，验证下 OOM 以及分析 Dump 文件。

设置 VM 参数：-Xms3m -Xmx3m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=./

```
public static void main(String[] args) {  
    List<Object> oomList = Lists.newArrayList();  
    // 无限循环创建对象  
    while (true) {  
        oomList.add(new Object());  
    }  
}
```

通过报错信息得知，`java heap space` 表示 OOM 发生在堆区，并生成了 hprof 二进制文件在当前文件夹下。

```
java.lang.OutOfMemoryError: Java heap space
Dumping heap to ./java_pid40550.hprof ...
Heap dump file created [5559357 bytes in 0.025 secs]
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.util.Arrays.copyOf(Arrays.java:3210)
    at java.util.Arrays.copyOf(Arrays.java:3181)
    at java.util.ArrayList.grow(ArrayList.java:265)
    at java.util.ArrayList.ensureExplicitCapacity(ArrayList.java:239)
    at java.util.ArrayList.ensureCapacityInternal(ArrayList.java:231)
    at java.util.ArrayList.add(ArrayList.java:462)
    at cn.acmenlt.demo.oom.OomMainTest.main(OomMainTest.java:28)
Disconnected from the target VM, address: '127.0.0.1:65044', transport: 'socket'
```

JvisualVM 分析

Dump 分析工具有很多，相对而言 JvisualVM、JProfiler、Eclipse Mat，使用人群更多一些。下面以 JvisualVM 举例分析 Dump 文件。

🕒 [heapdump] java_pid40550.hprof

堆 Dump

🔍 检查

📄 最大的对象:

查找 20 保留大小最大的对象 查找

类名	保留大小
java.lang.Object[]#793	2,561,064
class sun.util.calendar.ZoneInfoFile	182,710
sun.misc.Launcher\$AppClassLoader#1	135,639
byte[]#1	106,218
sun.misc.URLClassPath#2	74,963
class java.io.File	54,410
java.io.UnixFileSystem#1	53,994
java.io.ExpiringCache#2	52,681
java.io.ExpiringCache\$1#2	52,637
sun.misc.Launcher\$ExtClassLoader#1	32,775
java.lang.Object[]#752	29,378
class sun.util.resources.zh.CurrencyNames_zh_CN	29,378
java.util.Vector#5	29,085
java.lang.Object[]#776	29,049
java.lang.String[]#84	28,772
java.util.HashMap#9	25,774
java.util.HashMap\$Node[]#11	25,710
java.io.PrintStream#1	25,208
java.io.PrintStream#2	25,208

📄 基本信息:

生成的日期: Wed Sep 22 17:09:40 CST 2021
文件: /Users/single/Desktop/40550/java_pid40550.hprof
文件大小: 5.3 MB

字节总数: 3,586,297
类总数: 847
实例总数: 121,899
类加载器: 3
垃圾回收根节点: 697
等待结束的悬挂对象数: 0

在出现 OutOfMemoryError 异常错误时进行了堆转储
导致 OutOfMemoryError 异常错误的线程 **main** 这个很有用

📄 环境:

操作系统: Mac OS X (10.14.6)
体系结构: x86_64 64bit
Java 主目录: /Library/Java/JavaVirtualMachines/jdk1.8.0_241.jdk/Contents/Home/jre
Java 版本: 1.8.0_241
JVM: Java HotSpot(TM) 64-Bit Server VM (25.241-b07, mixed mode)
Java 供应商: Oracle Corporation

📄 系统属性:

[显示系统属性](#)

📄 堆转储上的线程:

[显示线程](#)

列举两个常用的功能，第一个是能看到触发 OOM 的线程堆栈，清晰得知程序溢出的原因。

概述

堆转储上的线程:

```
"main" prio=5 tid=1 RUNNABLE
  at java.lang.OutOfMemoryError.<init>(OutOfMemoryError.java:48)
  at java.util.Arrays.copyOf(Arrays.java:3210)
    Local Variable: class java.lang.Object\[\]
  at java.util.Arrays.copyOf(Arrays.java:3181)
    Local Variable: java.lang.Object\[\]#793
  at java.util.ArrayList.grow(ArrayList.java:265)
  at java.util.ArrayList.ensureExplicitCapacity(ArrayList.java:239)
  at java.util.ArrayList.ensureCapacityInternal(ArrayList.java:231)
  at java.util.ArrayList.add(ArrayList.java:462)
    Local Variable: java.lang.Object#29136
  at cn.acmenlt.demo.oom.OomMainTest.main(OomMainTest.java:28)
    Local Variable: java.util.ArrayList#50
    Local Variable: java.lang.String\[\]#141
```

第二个就是可以查看 JVM 内存里保留大小最大的对象，可以自由选择排查个数。

检查 ×

最大的对象:

查找 保留大小最大的对象:

类名	保留大小
java.lang.Object[]#793	2,561,064
class sun.util.calendar.ZoneInfoFile	182,710
sun.misc.Launcher\$AppClassLoader#1	135,639
byte[]#1	106,218
sun.misc.URLClassPath#2	74,963

点击对象还可以跳转具体的对象引用详情页面。

java.lang.Object[]				实例: 793 总大小: 945,032 保留大小: 2,745,733			
实例数				字段			
实例	大小	保留		字段	类型	值	保留
#793	853,704	2,561,064		this	Object[]	#793 106,710 个项	2,561,064
#1 1,024 个项	8,216	16,800		> <项 0-499>	Object	(500 项)	-
#759 664 个项	5,336	5,336		> <项 500-999>	Object	(500 项)	-
#752 426 个项	3,432	29,378		> <项 1,000-1,499>	Object	(500 项)	-
#412 241 个项	1,952	1,952		> <项 1,500-1,999>	Object	(500 项)	-
#763 216 个项	1,752	13,512		> <项 2,000-2,499>	Object	(500 项)	-
#411 212 个项	1,720	1,720		> <项 2,500-2,999>	Object	(500 项)	-
#776 160 个项	1,304	29,049		> <项 3,000-3,499>	Object	(500 项)	-
#642 150 个项	1,224	1,224		> <项 3,500-3,999>	Object	(500 项)	-
#750 106 个项	872	4,276		> <项 4,000-4,499>	Object	(500 项)	-
#657 81 个项	672	876		> <项 4,500-4,999>	Object	(500 项)	-
#124 80 个项	664	664					
#413 78 个项	648	648					
#125 73 个项	608	608					
#517 73 个项	608	608					
#742 72 个项	600	768					
#620 69 个项	576	636					
#707 66 个项	552	834					
#293 62 个项	520	794					
#313 60 个项	504	994					
#525 57 个项	480	562					
#617 56 个项	472	846					
#668 56 个项	472	1,090					
#602 50 个项	424	480					
引用				引用			
字段	类型	值	保留	字段	类型	值	保留
this (Java frame)	Object[]	#793 106,710 个项	2,561,064				
elementData (Java frame)	ArrayList	#50	32				

文中 Dump 文件较为简单，而正式环境出错的原因五花八门，所以不对该 Dump 文件做深度解析。

注意：JvisualVM 如果分析大 Dump 文件，可能会因为内存不足打不开，需要调整默认的内存。

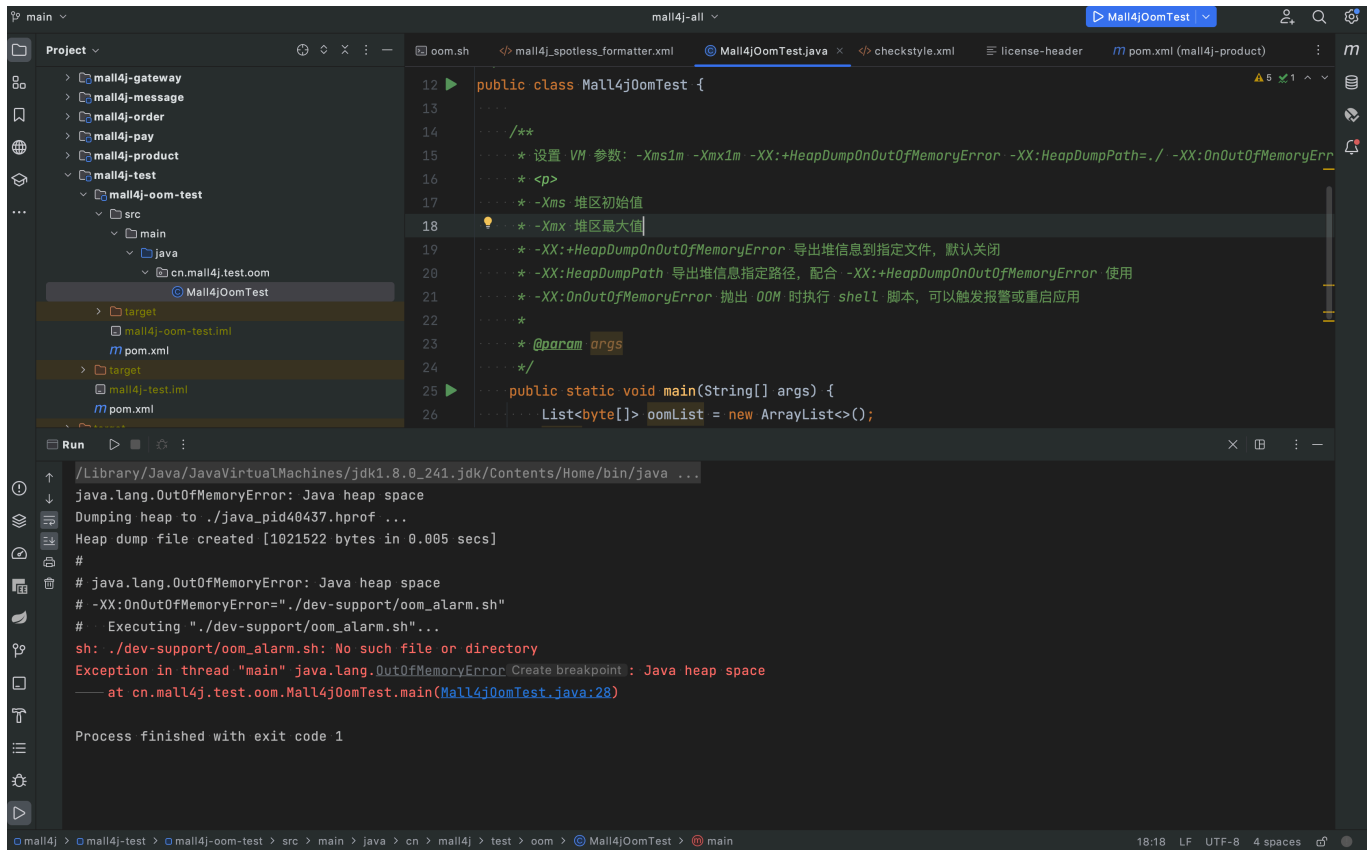
如何处理 OOM

JVM 启动参数配置添加以下参数：

- -XX:+HeapDumpOnOutOfMemoryError
- 导出堆信息到指定文件，默认关闭
- -XX:HeapDumpPath=./（参数为 Dump 文件生成路径）
- 导出堆信息指定路径，配合 -XX:+HeapDumpOnOutOfMemoryError 使用
- -XX:OnOutOfMemoryError=./dev-support/oom.sh
- 抛出 OOM 时执行 shell 脚本

-XX:OnOutOfMemoryError 参数很关键，可以做很多 OOM 触发后的个性化操作，重启应用或者通知开发解决问题。

查看星球 [刚果商城项目](#) `org.opengoofy.congomall.test.oom.CongoMall100MTest` 类测试代码，验证下 OOM。通过报错信息得知，java heap space 表示 OOM 发生在堆区，并生成了 hprof 二进制文件在当前文件夹下。



```
12 public class Mall4jOomTest {
13     ...
14     /**
15      * 设置 VM 参数: -Xms1m -Xmx1m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=./ -XX:OnOutOfMemoryErr
16      * <p>
17      * -Xms 堆区初始值
18      * -Xmx 堆区最大值
19      * -XX:+HeapDumpOnOutOfMemoryError 导出堆信息到指定文件，默认关闭
20      * -XX:HeapDumpPath 导出堆信息指定路径，配合 -XX:+HeapDumpOnOutOfMemoryError 使用
21      * -XX:OnOutOfMemoryError 抛出 OOM 时执行 shell 脚本，可以触发报警或重启应用
22      *
23      * @param args
24      */
25     public static void main(String[] args) {
26         List<byte[]> oomList = new ArrayList<>();
```

```
/Library/Java/JavaVirtualMachines/jdk1.8.0_241.jdk/Contents/Home/bin/java ...
java.lang.OutOfMemoryError: Java heap space
Dumping heap to ./java_pid40437.hprof ...
Heap dump file created [1021522 bytes in 0.005 secs]
#
# java.lang.OutOfMemoryError: Java heap space
# -XX:OnOutOfMemoryError="./dev-support/oom_alarm.sh"
# Executing "./dev-support/oom_alarm.sh"...
sh: ./dev-support/oom_alarm.sh: No such file or directory
Exception in thread "main" java.lang.OutOfMemoryError: Create breakpoint: Java heap space
--- at cn.mall4j.test.oom.Mall4jOomTest.main(Mall4jOomTest.java:28)

Process finished with exit code 1
```

并通过 `-XX:OnOutOfMemoryError=./dev-support/oom.sh` 指定在 OOM 时执行了某一 shell 脚本，向所在服务的群组触发了通知。

```
# 钉钉群里创建机器人
# 设置自定义关键字：报警
curl 'https://oapi.dingtalk.com/robot/send?
access_token=4acbf3675e8ec5923bb890d05c315bb4221ba4b9d4062c01b67939cfed02e612' \
-H 'Content-Type: application/json' \
-d '{"msgtype": "text","text": {"content": "应用触发 OOM 报警，请尽快处理"}}'
```

这里仅演示 Demo，公司生产环境可扩展相关执行方案。

文末总结

线上如遇到 JVM 内存溢出，可以分以下几步排查：

1. `jmap -heap` 查看是否内存分配过小;
2. `jmap -histo` 查看是否有明显的对象分配过多且没有释放情况;
3. `jmap -dump` 导出 JVM 当前内存快照, 使用 JDK 自带或 MAT 等工具分析快照。

如果上面还不能定位问题, 那么需要排查应用是否在不断创建资源, 比如网络连接或者线程, 都可能会导致系统资源耗尽。

更新: 2023-08-01 15:38:42

原文: <<https://www.yuque.com/magestack/12306/cpk7ga8acmpe5nm8>>